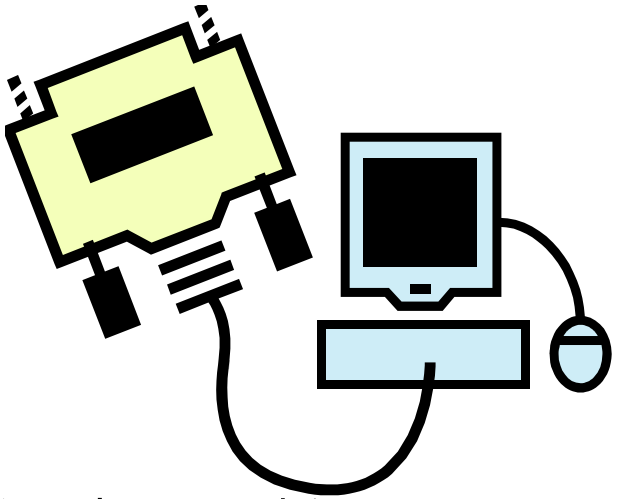
# マイコンシステム
# 第１４回

青森大学 ソフトウェア情報学部

橋本恭能

haship@aomori-u.ac.jp

# 目次



- （前回課題の解答例）
- 講義
  - 実装②
  - 関数の実装
    - カウントダウンを開始する・停止する、残り時間を更新する
- 例題
  - 設定時間・開始時間・残り時間の処理
- 課題
  - 状態遷移の動作確認②

# 講義

実装②

カウントダウンを開始する

カウントダウンを停止する

残り時間を更新する

# 組込みシステム開発

| 週 | テーマ | 内容 |
|---|---|---|
| 7 | キッチンタイマーの組立 | キッチンタイマーのハードを製作・確認 |
| 8<br>9 | 要求分析・外部設計 | キッチンタイマーに必要な機能を抽出<br>画面表示、ボタンを押した時の動作を設定<br>機能の実現手順をフローチャートで図示 |
| 10<br>11<br>12 | 内部設計 | 状態遷移図・状態遷移表の使い方<br>システムの内部構造を考える<br>機能の実現手順をより詳細に決める |
| 13<br>14 | 実装 | プログラムを作る |
| 15 | テスト・完成 | 設計通りにできているか動作検証する |

# カウントダウンを開始する

- いつ実行するか
  - 時間設定中からカウントダウン中へ遷移するとき
  - 一時停止中からカウントダウン中へ遷移するとき
- 処理内容

1. カウントダウンの開始時間を現時点に指定すること
   - プログラム実行後から経過時間を返す関数millis()を使い、変数startTimeに代入　`startTime = millis();`
2. 設定時間timeに入っている値を残り時間remainTimeに代入し、残り時間更新に備える

   `remainTime = time;`

# カウントダウンを停止する

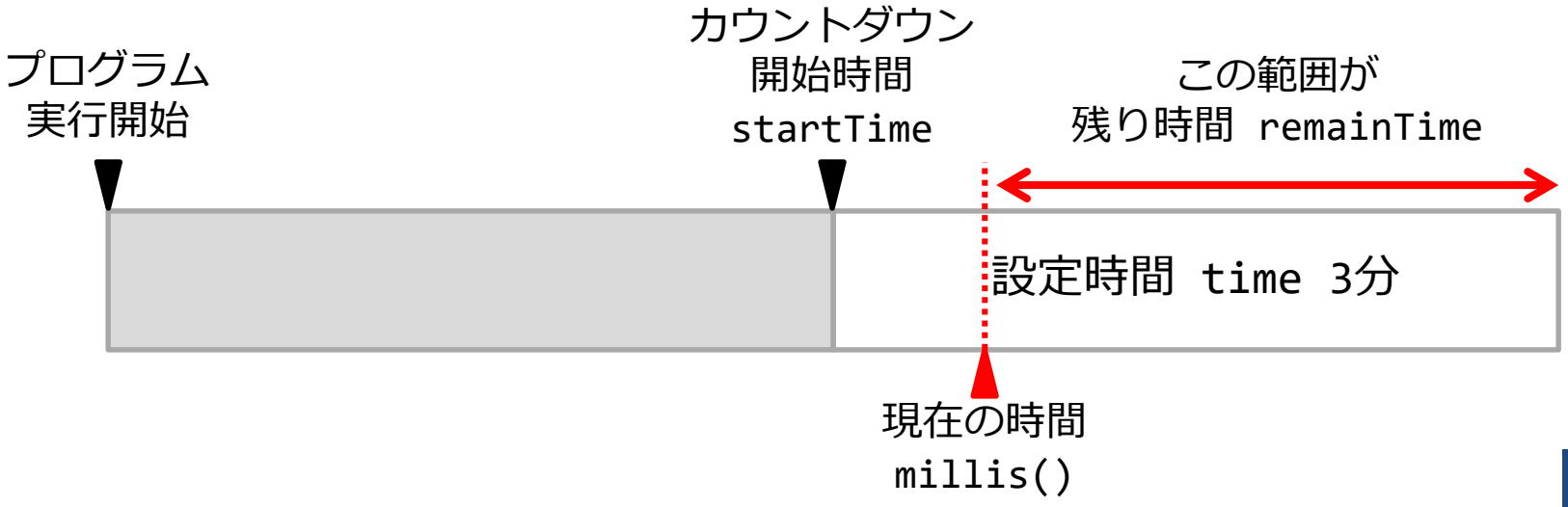
- いつ実行するか
  - カウントダウン中から一時停止中へ遷移するとき

- 処理内容
  1. カウントダウン中に減った残り時間remainTimeを、一時停止中に設定時間timeに戻す（さらに分ボタンで時間を増やす可能性があるため）

     ```
     time = remainTime;
     ```

# 残り時間を更新する

- カウントダウン中は常に実行する
- 処理内容
  - 以下の図を参考に考える
  - 残り時間を計算で求める
- 例）設定時間3分



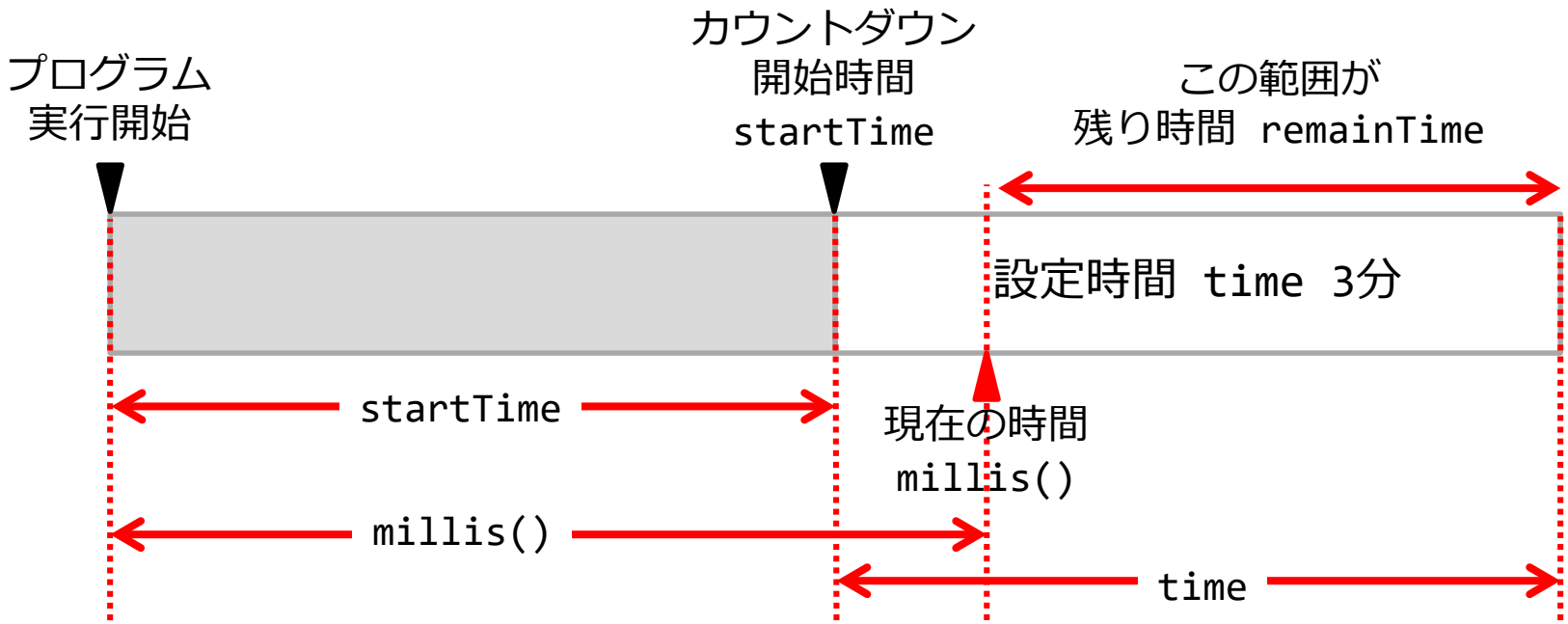
# 残り時間を更新する

- 残り時間＝（？）＋（？）－（？）

  remainTime = (_____) + (_____) - (_____) ;

- 選択肢

  設定時間 time，開始時間 startTime，現在の時間 millis()



# 例題

設定時間・開始時間・残り時間の
処理

# 例題のまとめ　メインタブ（完成）

```
//config.h
enum State {
  STATE_INIT,
  STATE_SETTING,
  STATE_COUNTDOWN,
  STATE_PAUSE,
  STATE_ALARM
} state;

enum Event {
  EVENT_NONE,
  EVENT_PUSH_MINUTE_BUTTON,
  EVENT_PUSH_START_STOP_BUTTON,
  EVENT_TIME_IS_UP
} event;
```

```
//KT2018_04
#include ”config.h”
long time;
long startTime;
long remainTime;
void setup() {
  setupDevice();
  time = 0L;
  startTime = 0L;
  remainTime = 0L;
  state = STATE_INIT;
  event = EVENT_NONE;
}
void loop() {
  event = receiveEvent();
  state = handleEvent(state, event);
}
```

# 例題のまとめ　Actionタブ（完成）

```
//Actionタブ
void startTimer(){
  //開始時間に現在の時間を代入する
  startTime = millis();

  //残り時間に設定時間を代入する
  remainTime = time;
}
void stopTimer(){
  //設定時間に残り時間を代入する
  time = remainTime;
}
void updateTimer(){
  //残り時間を計算する（課題）
  remainTime = ? + ? - ? ;
}
//以下省略（前回入力済み）
```

追加

追加

追加

3か所に
別々の値を
入れる

- 以下前回作業で完成のため省略
  - Deviceタブ
  - Eventタブ
  - Stateタブ

# 課題

状態遷移の動作確認②

すべての状態とイベントの

組み合わせを確認する

# 動作確認

- 状態遷移表を確認する

太枠内は
前回確認済み

| | 初期状態 | 時間設定中 | カウントダウン中 | 一時停止中 | アラーム鳴動中 |
|---|---|---|---|---|---|
| 分ボタンを押す | 計測時間を1分増加 | 計測時間を1分増加 | ／（無視） | 計測時間を1分増加 | ／（無視） |
| | 時間設定中 | 時間設定中 | | 時間設定中 | |
| Start/Stopボタンを押す | ／（無視） | カウントダウンを開始する | カウントダウンを停止する | カウントダウンを開始する | アラームを止める |
| | | カウントダウン中 | 一時停止中 | カウントダウン中 | 初期状態 |
| 残り時間が0になる | ／（無視） | ×（不可） | アラームを鳴らす | ×（不可） | ／（無視） |
| | | | アラーム鳴動中 | | |

# 状態遷移表のすべての組み合わせを確認

- アクションと遷移先がある箇所
  - その状態でイベントが起きたら
    アクションが実行されることを確認
  - アクションが実行された後、次の状態に遷移
    することを確認
  - 例）状態　　：初期状態
    　　イベント：分ボタンを押す
    - 設定時間が1分増加することを画面表示で確認
    - 1分増加した後、状態が時間設定中（STATE_SETTING)に遷移したこ
      とを画面表示で確認

# 状態遷移表のすべての組み合わせを確認

- 無視の箇所
  - その状態でイベントが発生したら
    **<u>何の処理も実行されない</u>**ことを確認
  - アクションが実行された後、
    **<u>状態遷移しない</u>**ことを確認
  - 例）状態　　：カウントダウン中
    　　イベント：分ボタンを押す
    - カウントダウン中は分ボタンを押しても、状態は変化しない、時間は増えない
  - もし、イベントを起こせない場合は無視から不可へ変更

# 状態遷移表のすべての組み合わせを確認

- 不可の箇所
  - その状態でイベントは起きない（起こせない）
  - ただし起こせる場合は、不可の指定は間違いになるので無視に変更
- 確認のまとめ
  - アクションと遷移先がある箇所：
    - アクションが実行されることを確認
    - 次の状態に遷移することを確認
  - 無視：何も起きない・遷移しないことを確認
    無視のままでよいか確認
  - 不可：不可のままでよいか確認

# 課題の提出方法

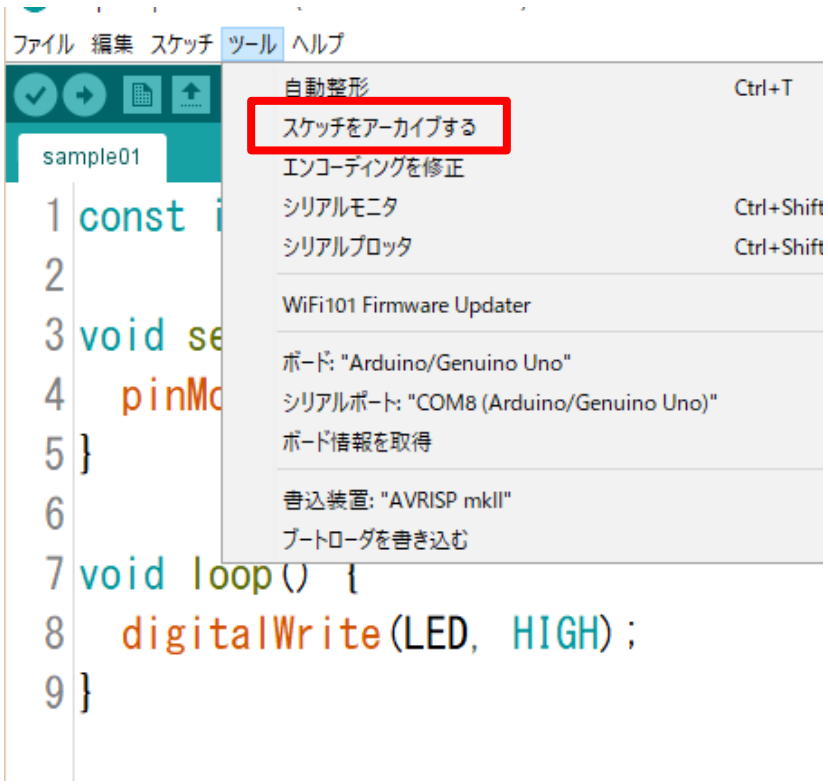
無視から不可

不可から無視

へ変更する箇所があるか

# プログラムをアーカイブする

- ツール ->
  スケッチをアーカイブ




- 圧縮ファイル名を指定

# 課題 提出先・期限

- 圧縮ファイルをメールに添付して提出する
  - 課題プログラム KT2018_04
  - 提出前に動作確認してから提出すること
- 提出先
  - haship@aomori-u.ac.jp
  - 件名：マイコンシステム14週
  - 本文に学籍番号、氏名を書く
- 提出期限
  - 次回授業前まで